

## Bachelor Thesis

Adimen Artifizialeko Gradua

---

# Evaluating the Impact of Network Topology on Backdoor Attacks in Decentralized Federated Learning

---

*Iker Bereziartua Sagarna*

### Advisors

Zuzendaria 1 / Director(a) 1

Zuzendaria 2 / Director(a) 2

April 28, 2026



# Esker onak / Agradecimientos

Eskerrak eman nahi izanez gero, hemen idatzi testua. En caso de querer añadir agradecimientos, escribir aquí el texto.

Atal hau nahi ez baduzu, *main.tex* fitxategian komentatu lerro hori. En caso de no querer este apartado, comentalo en el fichero *main.tex*.



# Laburpena / Resumen

Idatzi hemen laburpena. Escribe aquí el resumen.



# Contents

|                                                                      |            |
|----------------------------------------------------------------------|------------|
| <b>Contents</b>                                                      | <b>v</b>   |
| <b>List of Figures</b>                                               | <b>vi</b>  |
| <b>List of Tables</b>                                                | <b>vii</b> |
| <b>Índice de algoritmos</b>                                          | <b>ix</b>  |
| <b>1 Introduction</b>                                                | <b>3</b>   |
| <b>2 Theoretical Framework</b>                                       | <b>5</b>   |
| 2.1 Federated Learning . . . . .                                     | 5          |
| 2.2 Decentralized Federated Learning (Gossip Learning) . . . . .     | 7          |
| <b>3 Threat Landscape in Decentralized Learning</b>                  | <b>9</b>   |
| 3.1 Privacy Attacks . . . . .                                        | 9          |
| 3.2 Utility Attacks . . . . .                                        | 10         |
| 3.2.1 Data Poisoning Attacks . . . . .                               | 10         |
| 3.2.2 Model Poisoning Attacks . . . . .                              | 11         |
| 3.2.3 Backdoor Poisoning Attacks . . . . .                           | 11         |
| <b>4 Methodology</b>                                                 | <b>13</b>  |
| 4.1 Threat Model . . . . .                                           | 13         |
| 4.2 Backdoor Attack Implementation . . . . .                         | 13         |
| <b>5 Experimentation</b>                                             | <b>17</b>  |
| 5.1 Evaluated Network Topologies . . . . .                           | 17         |
| 5.2 Experimental Setup: Dataset, Architecture, and Metrics . . . . . | 20         |
| 5.2.1 Simulation Constraints and Execution Flow . . . . .            | 20         |
| <b>6 Experimental Results and Discussion</b>                         | <b>21</b>  |
| 6.1 Baseline Topologies Resilience . . . . .                         | 21         |
| 6.2 The Effectiveness vs. Stealthiness Trade-off . . . . .           | 23         |
| 6.3 Stochastic Block Model (SBM) Performance . . . . .               | 24         |

|                                                                 |           |
|-----------------------------------------------------------------|-----------|
| 6.3.1 The Adaptive Attacker: Navigating the Trade-off . . . . . | 25        |
| <b>Eranskina / ap ndice</b>                                     | <b>29</b> |

# List of Figures

|   |                                                                                                                                                                                                                                                         |    |
|---|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|----|
| 1 | Example of the backdoor trigger applied to the dataset. Left: A clean MNIST image. Right: A poisoned image with the $7 \times 7$ white patch applied to the bottom-right corner, maliciously relabeled to Class 0. . . . .                              | 14 |
| 2 | Theoretical visualization of the four network topologies evaluated in this study.                                                                                                                                                                       | 18 |
| 3 | Experimental setup for the deterministic Stochastic Block Model (SBM). The network is strictly divided into Community A and Community B, connected via a single gateway. The designated malicious node (Client 1) is placed within Community A. . . . . | 19 |
| 4 | Metric per Node in a Fully Connected topology . . . . .                                                                                                                                                                                                 | 22 |
| 5 | Metric per Node in a Ring topology . . . . .                                                                                                                                                                                                            | 22 |
| 6 | Metric per Node in a Star topology . . . . .                                                                                                                                                                                                            | 23 |
| 7 | Metric per Node in a SBM topology with a boosting factor of 5.0 . . . . .                                                                                                                                                                               | 24 |
| 8 | Metric per Node in a SBM topology with a boosting factor of 3.0 . . . . .                                                                                                                                                                               | 25 |
| 9 | Metric per Node in a SBM topology with a boosting factor of 4.0 . . . . .                                                                                                                                                                               | 26 |

# List of Tables

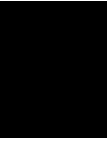
# Lista de Algoritmos

|     |                                                                            |   |
|-----|----------------------------------------------------------------------------|---|
| 2.1 | Standard Federated Averaging (FedAvg) . . . . .                            | 6 |
| 2.2 | Standard Decentralized Federated Learning (Gossip Learning) Loop . . . . . | 8 |



figurechapter tablechapter





# Introduction

The massive growth of Machine Learning in recent years has led to an unprecedented movement of data to centralized servers. Beyond creating severe bandwidth bottlenecks, this centralized approach raises significant privacy and regulatory concerns, as AI models frequently learn from highly sensitive or personal data. To address these challenges, Federated Learning (FL) was introduced [?]. The core philosophy of FL is to bring the model to the data rather than bringing the data to the model. In this architecture, a central server initializes a global model and distributes it to participating clients. Each client trains the model locally using its private dataset and then sends only the updated parameters back to the server for aggregation. This paradigm not only drastically reduces bandwidth consumption but also fundamentally preserves data privacy.

However, while traditional FL effectively solves the data privacy dilemma, its dependence on a central server introduces a Single Point of Failure (SPOF) and creates new communication bottlenecks as networks scale. To mitigate these structural weaknesses, the paradigm is evolving towards Decentralized Federated Learning (DFL), often implemented as Gossip Learning (GL) [?]. In DFL, the central orchestrator is eliminated entirely. Instead, clients communicate directly with their topological neighbors in a Peer-to-Peer (P2P) network, continuously exchanging and averaging model parameters to achieve a global consensus.

The removal of a central server, while improving fault tolerance and scalability, shifts the security issues entirely onto the network's topology. In centralized FL, the server can act as a robust gatekeeper, using the mass consensus of all the clients to dilute malicious updates. In DFL, the propagation of a poisoned model or a backdoor depends strictly on the structural layout of the network.

This structural shift exposes a critical research gap regarding how different network topologies influence the viability of localized poisoning attacks. In decentralized environments, an adversary attempting to compromise the global model faces a fundamental "Effectiveness vs. Stealthiness Trade-off" [?, ?]. To successfully spread a backdoor, an attacker must apply

## 1. INTRODUCTION

---

an aggressive attack to their malicious updates. However, if the attack is disproportionately aggressive relative to the network’s structural resistance, the attacker’s local neural network suffers from Catastrophic Forgetting. This destroys the node’s legitimate performance (Clean Accuracy), exposing the attacker and being easily detectable by standard anomaly detection systems.

This thesis aims to empirically demonstrate how network topology dictates the parameters and success rate of poisoning attacks in decentralized machine learning environments. The specific objective of this research are:

- To evaluate the vulnerability of distinct network topologies (Fully Connected, Ring, Star and Stochastic Block Model) to local backdoor attacks. mas general
- To analyze the trade-off between an attack’s Effectiveness (Attack Success Rate) and its Stealthiness (Clean Accuracy).
- To investigate the impact of attacker placement within the already mentioned topologies, specifically comparing the propagation dynamics when an adversary is located at a structural bottleneck (Gateway Attacker) versus deep within an isolated community (Peripheral Attacker) [?].
- To conduct a scalability analysis to determine how increasing the overall network size affects the natural dilution of malicious model weights in decentralized averaging.
- To validate the proposed threat models and topological defenses using a practical industrial dataset (NEU-DET) [?], bridging theoretical graph dynamics with real-world implementation scenarios.

The rest of this document is organized as follows: Chapter 2 establishes the theoretical framework, detailing the transition from centralized Federated Learning to Decentralized Federated Learning. Chapter 3 explores the threat landscape in decentralized environments, outlining the specific vulnerabilities, adversarial threat models, and the mechanics of localized poisoning and backdoor attacks. Chapter 4 outlines the experimental methodology, defining the threat model, the hyperparameter configurations and the specific network topologies evaluated. Chapter 5 presents the simulation results, providing a comparative analysis of the attacker’s performance across different architectural constraints. Finally, Chapter 6 summarizes the core findings and proposes future lines of research in decentralized network security.

# Theoretical Framework

The transition from centralized machine learning to decentralized architectures represents a fundamental shift in how data privacy and computational resources are managed. This chapter outlines the foundational concepts of Federated Learning (FL) and its evolution into Decentralized Federated Learning (DFL), commonly referred to as Gossip Learning (GL), providing the necessary background to understand the structural vulnerabilities evaluated in this research.

## 2.1 Federated Learning

pseudokodigoak

In traditional machine learning paradigms, training a robust model requires aggregating vast amounts of data into a single centralized repository or data center. While this approach benefits from direct access to the entire dataset, it introduces severe limitations, including high communication costs, significant storage requirements, and profound data privacy concerns.

To mitigate these issues, McMahan et al. [?] introduced Federated Learning (FL) by decentralizing the training data. The core principle of FL is to bring the model to the data rather than gathering the data in a central location. In a standard FL architecture, a central server orchestrates the learning process across a distributed network of clients (e.g., mobile devices, edge servers, or organizations). The training process operates in iterative communication rounds:

1. The central server initializes a global model and broadcasts its parameters to a subset of participating clients.
2. Each client trains the model locally using its own private data.

## 2. THEORETICAL FRAMEWORK

---

### Centralized Training Loop

---

```

1 input: Total rounds  $T$ , participating clients  $K$ , local epochs  $E$ 
2 Server Execution:
3 Initialize global model  $w_0$ 
4 for  $t = 1$  to  $T$ 
5   Server selects a subset of clients  $S_t$ 
6   for each client  $k \in S_t$  in parallel do
7      $w_{t+1}^k \leftarrow \text{ClientUpdate}(w_t, k, E)$ 
8   done
9   // Server aggregates local models
10   $w_{t+1} \leftarrow \sum_{k \in S_t} \frac{n_k}{n} w_{t+1}^k$ 
11 rof
12
13 ClientUpdate( $w, k, E$ ):
14 Initialize local model  $w^k \leftarrow w$ 
15 for  $e = 1$  to  $E$ 
16   Train  $w^k$  on local data  $\mathcal{D}_k$ 
17 rof
18 return  $w^k$ 

```

---

**Algoritmo 2.1:** Standard Federated Averaging (FedAvg)

3. The clients compute the model updates (gradients or new weights) and transmit only these parameters back to the central server.
4. The server aggregates the local updates to form a new global model.

The most standard aggregation algorithm used in this process is Federated Averaging (FedAvg), where the server computes a weighted average of the received models based on the number of local training samples at each client. Mathematically, the global model  $w$  at round  $t + 1$  is updated as follows:

$$w_{t+1} = \sum_{k=1}^K \frac{n_k}{n} w_{t+1}^k \quad (2.1)$$

Where  $K$  is the number of participating clients,  $n_k$  is the number of data samples at client  $k$ ,  $n$  is the total number of samples across all participating clients, and  $w_{t+1}^k$  represents the local model weights of client  $k$ .

To accommodate the varying nature of data distribution in real-world applications, Yuan et al. [?] classify FL into three primary categories:

- **Horizontal Federated Learning (HFL):** Applied when clients share the same feature space but differ in data samples (e.g., two different hospitals predicting the same disease with different patients).
- **Vertical Federated Learning (VFL):** Applied when clients share the same sample ID space but differ in feature spaces (e.g., a bank and an e-commerce platform collaborating on the same user base).
- **Federated Transfer Learning (FTL):** Utilized when datasets differ in both samples and feature spaces, leveraging transfer learning techniques to overcome the lack of overlapping data.

Despite its advantages in preserving data privacy, FL remains reliant on a central server. This centralized coordination introduces a Single Point of Failure (SPOF) and creates a significant communication bottleneck as the network scales to thousands or millions of devices.

## 2.2 Decentralized Federated Learning (Gossip Learning)

To address the scalability limits and the SPOF inherent in centralized FL, researchers such as Hegedűs et al. [?] and Vanhaesebrouck et al. [?] proposed Decentralized Federated Learning (DFL). Often referred to as Gossip Learning (GL), this architecture removes the central orchestrator entirely. In a GL system, the network operates on a Peer-to-Peer (P2P) basis, where nodes communicate solely with their topological neighbors.

In this paradigm, there is no global synchronization or central authority to validate updates. Each node  $i$  maintains its own local model  $w_i$ . During the training process, nodes periodically exchange their model parameters with a subset of neighboring nodes, defined by the network's structural topology (e.g., a Ring, Star, or Stochastic Block Model). When a node receives models from its neighbors, it aggregates them with its own local parameters, typically using a weighted average, and then resumes local training.

The aggregation at a given node  $i$  at iteration  $t$  can be formally expressed as:

$$w_{i,t} = \sum_{j \in \mathcal{N}_i} \alpha_{ij} w_{j,t-1} \quad (2.2)$$

Where  $\mathcal{N}_i$  represents the set of neighbors of node  $i$  (including node  $i$  itself), and  $\alpha_{ij}$  denotes the aggregation weight assigned to the model received from node  $j$ .

This fully decentralized approach offers superior fault tolerance, as the network can seamlessly handle node dropouts and dynamic connectivity without halting the learning process. Furthermore, by eliminating the central server, GL mitigates the risk of a single entity controlling or monitoring the entire aggregation process.

To formalize this decentralized architecture, Algorithm 2.2 outlines the standard peer-to-peer training lifecycle. Unlike centralized systems that wait for a global synchronization

## 2. THEORETICAL FRAMEWORK

---

### Decentralized Training Loop

---

```

1 input: Graph  $\mathcal{G} = (\mathcal{V}, \mathcal{E})$ , Local datasets  $\mathcal{D}_i$ , epochs  $E$ 
2 Initialization: Each node  $i$  initializes local model  $w_{i,0}$ 
3 for  $t = 1$  to  $T$ 
4   for each node  $i \in \mathcal{V}$  in parallel do
5     // Phase 1: Local Training
6      $w_{i,t}^{local} \leftarrow \text{LocalTraining}(w_{i,t-1}, \mathcal{D}_i, E)$ 
7     // Phase 2: Gossip Communication
8     Send  $w_{i,t}^{local}$  to all topological neighbors
9     Receive  $w_{j,t}^{local}$  from all topological neighbors
10    // Phase 3: Decentralized Aggregation
11     $w_{i,t} \leftarrow \sum_{j \in \mathcal{N}_i} \alpha_{ij} w_{j,t}^{local}$ 
12  done
13 rof

```

---

**Algoritmo 2.2:** Standard Decentralized Federated Learning (Gossip Learning) Loop

barrier, each node in a Gossip Learning environment independently executes its local training, broadcasts its updates to its topological neighborhood, and computes its own localized consensus.

However, the absence of a central server fundamentally shifts the security dynamics of the system. Without an authoritative entity to establish trust, verify updates, and enforce a global consensus, the propagation of information depends entirely on the physical or logical shape of the network graph. This structural dependence means that model convergence, as well as the spread of malicious updates, is dictated by the network topology. Denser networks may facilitate rapid knowledge sharing but also accelerate the spread of poisoned parameters, whereas sparse networks or community-based structures may introduce severe communication bottlenecks. Understanding these topological dynamics is crucial for evaluating the vulnerabilities of GL systems against targeted adversarial attacks.

To formalize this decentralized architecture, Algorithm 2.2 outlines the standard peer-to-peer training lifecycle. Unlike centralized systems that wait for a global synchronization barrier, each node in a Gossip Learning environment independently executes its local training, broadcasts its updates to its topological neighborhood, and computes its own localized consensus.

# Threat Landscape in Decentralized Learning

Decentralized Federated Learning improves scalability and privacy by removing the need for a central server. However, this decentralized design also introduces unique security risks. Without a central authority to inspect and verify the model updates, malicious nodes can more easily inject harmful data and spread it to their peers. The following sections will dive deeper into the specific security vulnerabilities of Decentralized Federated Learning, categorizing the attacks into Privacy Attacks and Utility Attacks.

## 3.1 Privacy Attacks

One of the primary concerns in Decentralized Federated Learning is the risk of privacy attacks. Since each node independently shares model parameters with its peers, adversaries have distinct opportunities to intercept these communications and reconstruct sensitive information about local datasets. Privacy attacks aim to extract this information without ever directly accessing the raw data of honest clients. Because the parameters and gradients in Gossip Learning are derived directly from private data, analyzing these updates over multiple communication rounds can reveal substantial information about individual participants.

The most common method for extracting this data is through Inference Attacks, which exploit patterns in model updates to deduce sensitive properties. These attacks can take several forms. Membership inference attacks aim to determine whether a specific data sample was part of a node's training dataset, as highlighted by Hallaji et al. [?], these attacks take advantage of the fact that models often behave differently when encountering data they observed during training compared to unseen data. Class representation inference attacks, on the other hand, attempt to reconstruct the general characteristics or statistical properties of data belonging to a specific class, such as deducing transaction patterns indicative of fraudulent

behavior. Finally, property inference attacks seek to learn sensitive attributes about the training dataset, even if those attributes are not explicitly used for the classification task itself.

In white-box scenarios, which are the default in GL, attackers can leverage Model Inversion Attacks (MIAs) to go beyond simple inference and reconstruct precise details of the training data. Han et al. [?] demonstrate that by utilizing optimization techniques like gradient ascent, an attacker can iteratively adjust a dummy input to maximize the model’s output confidence, essentially reverse engineering the raw data that generated the intercepted gradients. Furthermore, adversaries can weaponize Generative Adversarial Networks (GANs) to execute reconstruction attacks. Research by Bouacida and Mohaisen [?], as well as broader surveys by Xie et al. [?], illustrates how an attacker can train a GAN on intercepted features where the attacker pits a generator against a discriminator until the generator successfully learns to produce highly realistic, synthetic samples that closely resemble the private training data of benign clients.

Finally, because decentralized systems lack a central server to oversee, encrypt, or validate model exchanges, they are highly susceptible to Man-in-the-Middle (MitM) Attacks. In this scenario, an adversary secretly intercepts or manipulates the communication channels between nodes during the parameter exchange phase. Bouacida and Mohaisen [?] warn that by eavesdropping on these updates, an attacker can silently extract sensitive information using the inference and inversion techniques detailed above, making MitM attacks exceptionally difficult to detect in a peer-to-peer architecture.

## 3.2 Utility Attacks

Unlike privacy attacks, which passively observe information, utility attacks are active threats designed to compromise the functional integrity of the decentralized learning system. As categorized by Xia et al. [?], these scenarios involve malicious nodes intentionally injecting misleading parameters or corrupting model updates to degrade the global model’s accuracy, prevent convergence, or force the network to learn targeted malicious behaviors. Because Decentralized Federated Learning (DFL) relies entirely on peer-to-peer consensus without a central server to filter anomalous updates, isolating these malicious actors is a complex challenge. Utility attacks are generally executed through poisoning strategies.

### 3.2.1 Data Poisoning Attacks

Data poisoning attacks occur during the local data collection or preparation phase. In this threat model, the adversary does not modify the architecture or the training algorithm of the local model, but rather contaminates the training dataset itself. These attacks generally fall into two categories:

- **Clean-Label Attacks:** The attacker subtly manipulates the input features of the training samples (e.g., adding imperceptible noise) while leaving the assigned labels unchanged. This approach is highly stealthy and is designed to bypass systems that employ strict data or label verification mechanisms.

- **Dirty-Label Attacks:** The attacker maliciously alters the labels of the training data (e.g., systematically flipping the label of a specific class to another). As noted by Xia et al. [?], training on incorrectly labeled data forces the local model to learn false statistical associations, which are subsequently encoded into its parameters and propagated to neighboring nodes during the exchange phase.

### 3.2.2 Model Poisoning Attacks

A more direct and mathematically potent threat in DFL is model poisoning. Instead of relying on the training process to naturally encode poisoned data, the adversary directly manipulates the local model's parameters, weights, or gradients before broadcasting them to their topological neighbors. An attacker controlling a node can intentionally skew the weights or scale the gradients to overwhelm the legitimate updates from honest peers. Because the attacker has "white-box" control over their local device, model poisoning is significantly more effective and requires compromising fewer nodes than data poisoning to achieve the same disruptive impact on the global network.

### 3.2.3 Backdoor Poisoning Attacks

A highly sophisticated subcategory of poisoning is the backdoor attack (also known as a targeted poisoning attack), foundational work which was established by Bagdasaryan et al. [?]. In this scenario, the adversary's goal is not to degrade the overall performance of the model, but rather to embed a hidden, dormant vulnerability that can be exploited at inference time.

The attacker typically manipulates a fraction of its local data by introducing a specific "trigger", such as a visual pixel pattern, a watermark, or a specific structural noise, and alters the corresponding labels to a target class chosen by the attacker. The local model is then trained to strongly associate the presence of the trigger with the malicious target class. The danger of a backdoor attack lies in its stealthiness: when deployed, the poisoned model will function completely normally on clean, benign inputs, maintaining a high *Clean Accuracy*. However, whenever the model processes an input containing the specific trigger, it will systematically misclassify it into the target class, yielding a high *Attack Success Rate (ASR)*.

**The Effectiveness vs. Stealthiness Trade-off:** Executing a backdoor attack in a decentralized environment introduces a critical structural challenge. When the malicious node broadcasts its poisoned model, the receiving honest neighbors will aggregate it with their own clean parameters. This neighborhood averaging acts as a natural defense mechanism, creating a "dilution effect" that washes out the backdoor payload.

To ensure the backdoor survives this decentralized aggregation and successfully infects the broader network, adversaries employ *Model Boosting* techniques. Both Bagdasaryan et al. [?] and Yang et al. [?] emphasize that by artificially amplifying the malicious weight differences by a scalar factor (the boosting factor) before transmission, the attacker attempts to force the backdoor into the local consensus of its neighbors.

However, this amplification introduces a severe dilemma. If the attacker applies a boosting factor that is too aggressive, the excessive distortion of the parameters will cause the local neural network to suffer from Catastrophic Forgetting, a phenomenon extensively studied by French [?]. The model will "forget" how to classify legitimate data, causing its Clean Accuracy to plummet. This performance collapse immediately exposes the malicious node as an outlier to standard anomaly detection systems. Conversely, if the boosting factor is too low, the network's topology will simply dilute and neutralize the attack. Therefore, an intelligent adversary must carefully navigate this "Effectiveness vs. Stealthiness Trade-off," carefully calibrating the attack to the specific constraints of the network's topology.

# Methodology

The methodology of this research is designed to evaluate the vulnerabilities of Decentralized Federated Learning by simulating targeted backdoor attacks. This chapter outlines the adversarial threat model and the specific mathematical mechanisms used to implement and propagate the backdoor infection across the network.

## 4.1 Threat Model

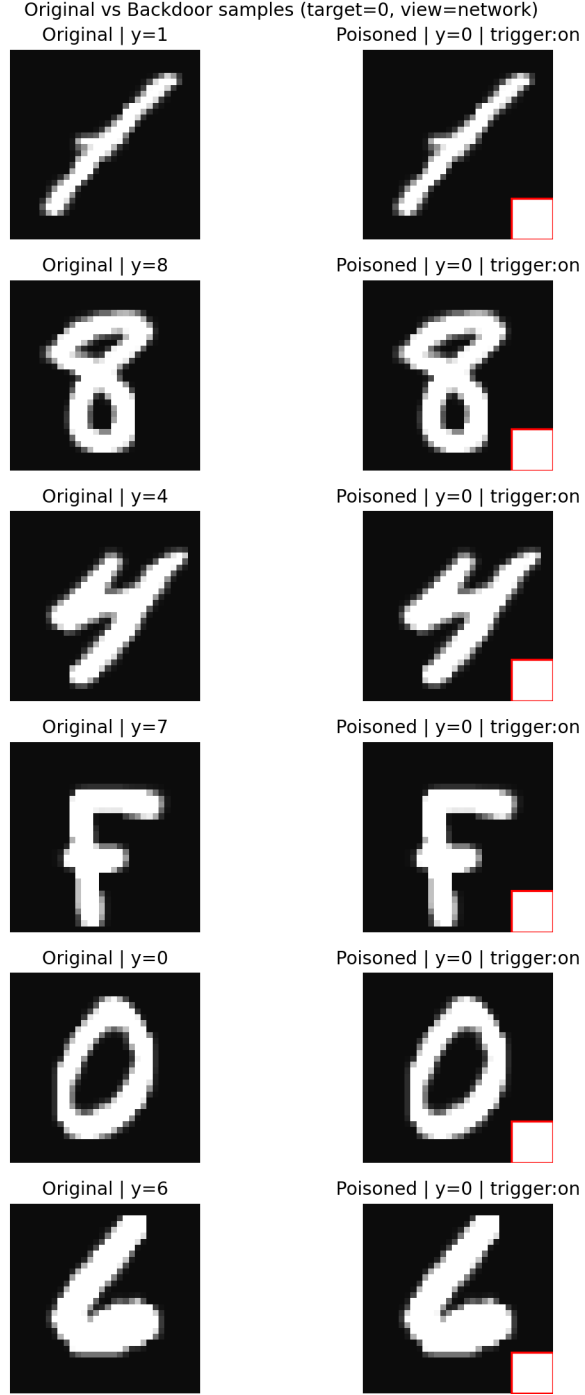
We assume a *Local White-Box* threat model. In this scenario, the attacker has full control over their designated local node. This grants the adversary complete access to manipulate their local training dataset, alter the local model’s architecture or hyperparameters, and inspect or modify the model’s weights before transmission. However, the attacker operates under strict decentralized constraints: they cannot directly manipulate the communication protocols, alter the global network graph, or interfere with the local aggregation functions and training processes of the honest peers.

## 4.2 Backdoor Attack Implementation

To evaluate the resilience of the decentralized network, the designated adversary (Client 1) executed a targeted backdoor poisoning attack.

The adversary manipulated 50% of its assigned local training dataset by applying a visual trigger. As shown in Figure 1, the trigger consists of a  $7 \times 7$  white square patch (maximum pixel intensity of 1.0) applied to the bottom-right corner of the selected images. The labels of these patched images were systematically altered to the target class (Class 0).

To guarantee that the malicious update survived the decentralized aggregation process, we applied the Model Boosting technique originally proposed by Bagdasaryan et al. [?]. Prior to broadcasting its local parameters to its topological neighbors, the malicious client



**Figure 1:** Example of the backdoor trigger applied to the dataset. Left: A clean MNIST image. Right: A poisoned image with the  $7 \times 7$  white patch applied to the bottom-right corner, maliciously relabeled to Class 0.

artificially scaled the parameter weight differences using a boosting factor ( $\alpha$ ), set aggressively to  $\alpha = 5.0$  in the baseline experiments:

$$Boosted_p = Global_p + \alpha \times (Local_p - Global_p) \quad (4.1)$$

By amplifying the weights, the attacker attempts to overpower the legitimate updates from honest peers, forcing the backdoor into the local consensus.



# Experimentation

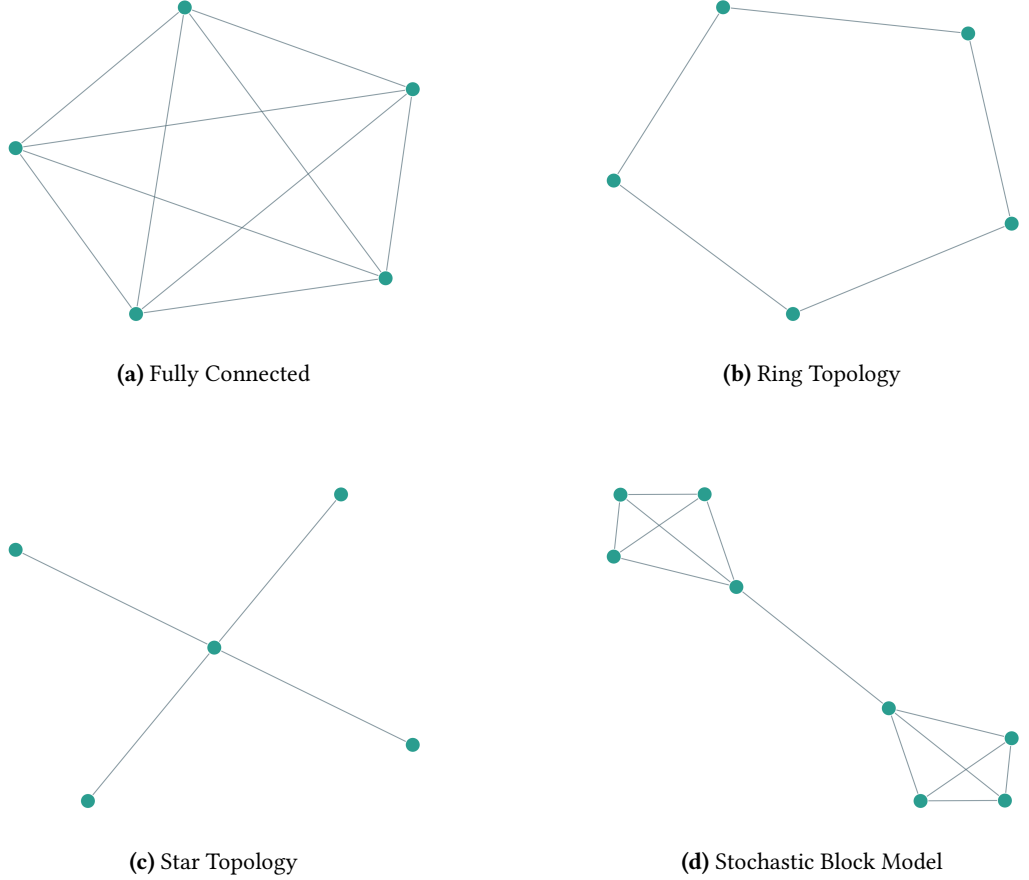
The main objective of this experimental setup is to explore the vulnerabilities explained previously, specifically backdoor attacks. By simulating a targeted backdoor attacks across different network structures, this research aims to understand how the lack of centralization affects the security of the learning process and how different communication topologies might naturally slow down or accelerate the spread of this type of attacks. All experiments were done using a custom Flower environment [?]. The source code and configuration files used to reproduce all the experiments are publicly available at: <https://github.com/IkerMardure/tfg-gossip-learning-backdoor/tree/master/GLow-master>

## 5.1 Evaluated Network Topologies

In Decentralized Federated Learning, the communication infrastructure is modeled mathematically as a graph  $\mathcal{G} = (\mathcal{V}, \mathcal{E})$ , where  $\mathcal{V}$  represents the set of participating nodes (clients) and  $\mathcal{E}$  represents the set of communication edges (links) between them [?]. Because nodes only aggregate models from their direct neighbors, the structural properties of  $\mathcal{G}$  dictate the flow of parameters, the speed of global convergence, and the network's inherent resilience to malicious injections.

To systematically evaluate these dynamics and establish a comprehensive baseline, decentralized networks were analyzed across a spectrum of connectivity, ranging from highly sparse to fully dense structures. The four foundational topologies evaluated in this research are visualized theoretically in Figure 2 and defined as follows:

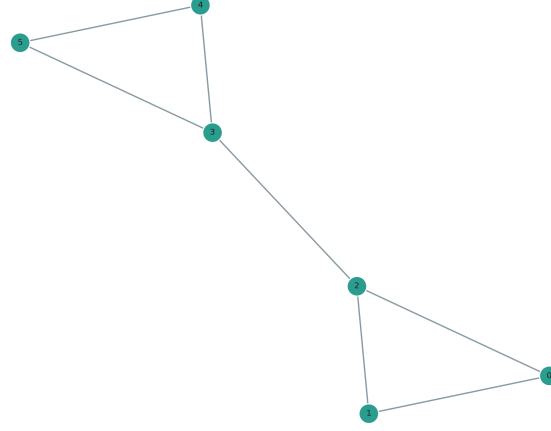
- **Fully Connected Topology:** A Fully Connected (FC) or complete graph is a topology where every node shares a direct edge with every other node in the network (Figure 2a). The topological distance (network diameter) is strictly one hop. This structure represents the maximum possible density in a peer-to-peer network, allowing for



**Figure 2:** Theoretical visualization of the four network topologies evaluated in this study.

immediate, unimpeded dissemination of knowledge. In these experiments, it is expected that denser networks will accelerate the backdoor attack due to direct peer-to-peer exposure.

- **Ring Topology:** A Ring topology represents a highly sparse, constrained network configuration (Figure 2b). Each node is connected to exactly two immediate neighbors, forming a single closed loop where information travels slowly [?]. Because there are no cross-network shortcuts, information must propagate sequentially. This structural bottleneck significantly delays the spread of both legitimate knowledge and potential adversarial payloads, naturally delaying the backdoor infection.
- **Star Topology:** A Star topology is characterized by a single central node (the hub) connected to multiple peripheral nodes, while the peripheral nodes remain completely isolated from one another (Figure 2c). In a DFL context, this localizes the centralized FL paradigm. The central hub acts as a powerful aggregator, continuously absorbing and



**Figure 3:** Experimental setup for the deterministic Stochastic Block Model (SBM). The network is strictly divided into Community A and Community B, connected via a single gateway. The designated malicious node (Client 1) is placed within Community A.

averaging updates from all peripherals to create a strong "consensus wall" capable of diluting anomalous updates originating from the periphery.

- **Stochastic Block Model (SBM):** Real-world decentralized networks rarely follow perfectly uniform shapes; instead, they naturally form communities. To mathematically capture this, Holland et al. [?] introduced the Stochastic Block Model, where nodes are partitioned into distinct blocks with a higher probability of intra-community edges than inter-community edges. This creates dense local clusters connected by sparse "gateway" links, trapping information in local echo chambers.

#### Deterministic SBM Implementation

It is important to note that while the theoretical Stochastic Block Model [?] is a probabilistic framework relying on random edge generation, our experimental methodology requires a controlled, deterministic environment to accurately isolate and measure the attack's propagation.

Therefore, rather than using random connections, we implemented a strictly deterministic community-based structure that mathematically mirrors the core characteristics of an SBM. As shown in Figure 3, the implemented network consists of two distinct communities containing three nodes each. Within each community, the nodes are fully connected to ensure rapid local consensus. However, the two communities are bridged by exactly one fixed gateway connection. This deterministic design guarantees that if a malicious node operates within one community, the gateway acts as a strict, quantifiable bottleneck, actively hindering the infection's progression to the external community.

## 5.2 Experimental Setup: Dataset, Architecture, and Metrics

The primary experiment was conducted using the MNIST dataset [?]. To align with the architectural requirements of the model, all the images were resized to 32x32 pixels. The classification task was constrained to a subset of 5 classes, primarily to lower computational cost. To establish a controlled and uniform baseline for the experiments, the dataset was partitioned in an Independent and Identically Distributed (IID) manner among the clients. This ensures that each node starts with a statistically similar subset of the data.

A modified *LeNet* [?] architecture was used as the local model across all nodes. The network is composed of three convolutional layers of 16, 32 and 64 filters, respectively, each with  $3 \times 3$  kernels and a padding of 1. Each convolutional operation is followed by a ReLU activation function and a  $2 \times 2$  Max Pooling operation. The resulting feature maps are processed through two fully connected layers, the first containing 128 units and the last one containing as much units as the classes used, in this case 5. Furthermore, a dropout layer with a 0.2 probability was inserted prior to the final classification layer to mitigate overfitting during the training. The global training process spanned 20 communication rounds, during which the active client performed local training for 2 epochs using a batch size of 128. The local optimization was handled by the Adam optimizer [?] with a learning rate set to 0.0001 and a momentum of 0.9.

To evaluate the attack, two measurements were used: Clean Accuracy and Attack Success Rate (ASR). Clean Accuracy measures how well each node classifies the unmodified test data, which is critical for observing how undetectable the attack remains during normal operation. In contrast, the ASR is calculated using modified test data to indicate the spread and effectiveness of the attack through the network.

### 5.2.1 Simulation Constraints and Execution Flow

While the theoretical framework of Gossip Learning assumes parallel execution across all participating nodes (as outlined in Algorithm 2.2), simulating such a decentralized network on centralized hardware introduces practical implementation constraints.

In our experimental setup using the Flower framework, the local training phases for the clients were executed sequentially within each communication round due to computational resource limits. However, from a logical standpoint, the network state is frozen during the training phase, and parameter exchanges are only aggregated at the end of each discrete round. This effectively simulates a synchronous, parallel execution environment. Because the primary focus of this research is the structural propagation of malicious weights through specific graph topologies rather than evaluating computational time delays or asynchronous race condition, this sequential simulation accurately preserves the integrity of the topological vulnerabilities being evaluated.

# Experimental Results and Discussion

This chapter presents the experimental results and evaluates the behavior of the decentralized learning system under a targeted backdoor attack. The analysis focuses on how different network architectures affect both the learning of legitimate data and the propagation of the malicious infection. The evaluation is divided into several phases: establishing the baseline resilience of standard topologies, analyzing the attacker's trade-off between effectiveness and stealthiness, evaluating community-based structures (SBM), and testing the system under scaled and more realistic conditions.

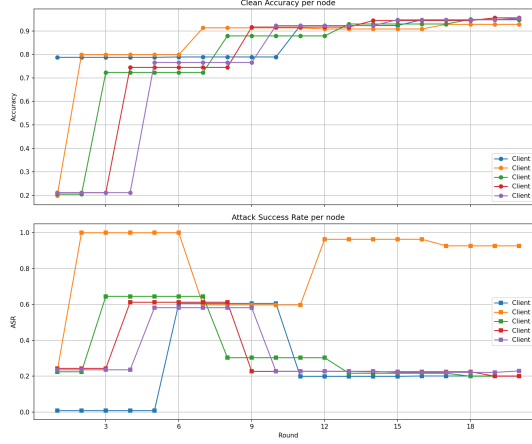
## 6.1 Baseline Topologies Resilience

The initial phase of evaluation analyzed the propagation of the backdoor attack across three fundamental communication structures: Fully Connected, Star, and Ring. All baseline experiments were conducted with a 5 node network over 20 communication rounds, utilizing a poisoning ratio of 50% and a boost factor of 5.0. To simulate a realistic environment where nodes already possess functional knowledge, a mixed initialization approach was used, starting the network with a base Clean Accuracy of approximately 70% to 80%.

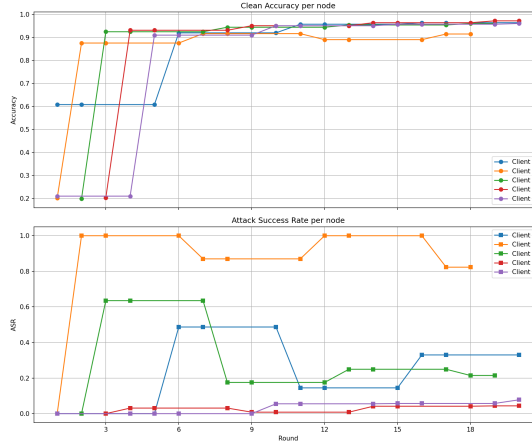
In the Fully Connected topology, as shown in Figure 4, the lack of communication barriers allowed the malicious updates to be broadcast widely during the initial phases. The attacker (Client 1) successfully pushed the Attack Success Rate (ASR) to the honest nodes to a peak of approximately 0.65 in the early rounds. However, the extended 20-round simulation revealed the long-term resilience of Gossip Learning. As the honest majority continued to train on clean data and broadcast their updates, the poisoned weights were progressively diluted. By round 10, the network "healed" itself through majority consensus, dropping the ASR back to 0.20 (the statistical equivalent of random guessing for 5 classes), while maintaining a high Clean Accuracy of over 0.90 across all nodes.

In contrast, the Ring topology, as shown in Figure 5, demonstrated a strong structural

## 6. EXPERIMENTAL RESULTS AND DISCUSSION



**Figure 4:** Metric per Node in a Fully Connected topology



**Figure 5:** Metric per Node in a Ring topology

resistance to rapid propagation [?]. Due to the strict neighborhood constraints, the backdoor was forced to travel sequentially. This structural bottleneck significantly delayed and weakened the attack. Unlike the Fully Connected network, the Ring prevented network-wide ASR spikes, keeping the infection contained while allowing the honest nodes to successfully improve their legitimate accuracy.

Finally, the Star topology exhibited a unique defense mechanism against peripheral attackers (see Figure 6). The results show that while the malicious node attempted to spread the backdoor, the central hub (Client 0) acted as an effective shield. Because the hub continuously averaged the poisoned weights with clean updates from the other three isolated peripheral nodes, the malicious parameters were heavily diluted. The central hub experienced a minor ASR increase (peaking near 0.40), but the rest of the peripheral nodes maintained an ASR of 0.0 throughout the entire simulation, effectively neutralizing the spread of the infection.

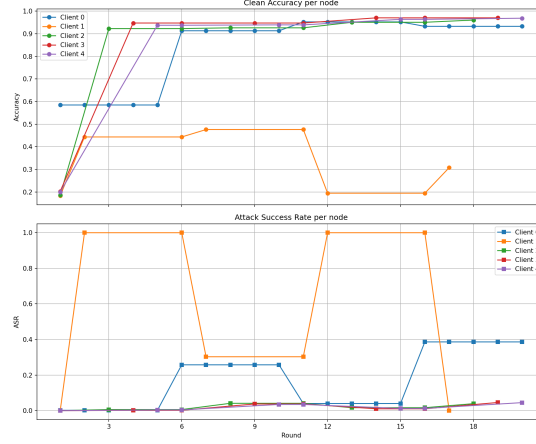


Figure 6: Metric per Node in a Star topology

## 6.2 The Effectiveness vs. Stealthiness Trade-off

The baseline results reveal a critical dynamic regarding the "Stealthiness" of the malicious node. While the primary goal of the adversary is to maximize the ASR across the network, this must be achieved without suffering from Catastrophic Forgetting [?]. In a realistic Gossip Learning environment, an attacker that experiences a severe drop in Clean Accuracy becomes an easily detectable outlier and risks immediate isolation by anomaly detection systems.

Our configuration proved highly stealthy in both the Fully Connected and Ring topologies. Despite using an aggressive boost factor of 5.0 and a 50% poisoning ratio, the attacker maintained a Clean Accuracy on par with the honest nodes (above 0.90). The attack was surgically assimilated locally without destroying the legitimate features of the model.

However, the Star topology exposed a critical vulnerability in the attacker's strategy when facing structural bottlenecks. As observed in the Star topology performance graph, the attacker (Client 1) suffered a complete collapse in its Clean Accuracy, plummeting to nearly 0.20. By repeatedly applying a massive boost factor of 5.0 against the strong, clean consensus wall provided by the central hub, the attacker's local optimization process was overwhelmed. The neural network's parameters became deformed in a futile attempt to force the backdoor into a highly resistant global model.

This demonstrates that stealthiness is not merely a product of the local hyperparameters (poisoning ratio and boost factor), but is also dependent on the network's topology. An aggressive attack configuration that remains perfectly hidden in a Fully Connected network can lead to the attacker's own Catastrophic Forgetting when isolated in a highly centralized structure like a Star network.

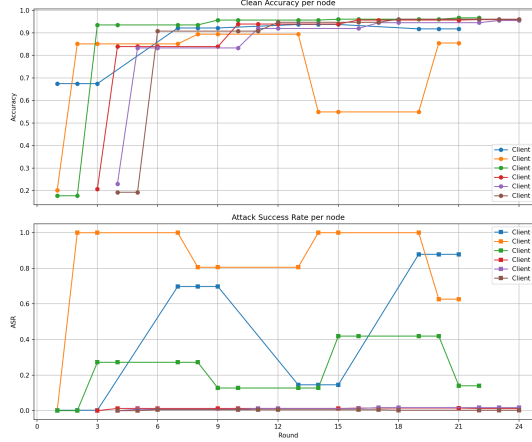


Figure 7: Metric per Node in a SBM topology with a boosting factor of 5.0

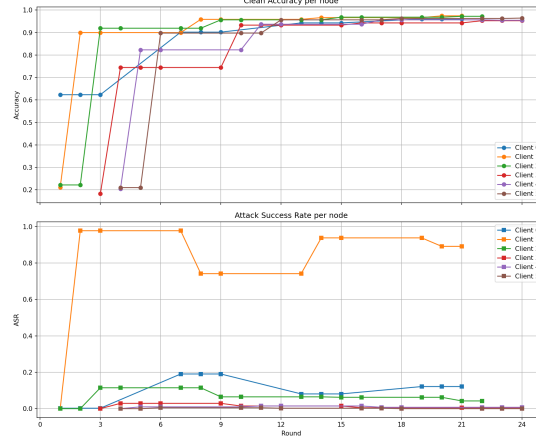
### 6.3 Stochastic Block Model (SBM) Performance

To evaluate the resilience of more realistic, community-based decentralized networks, the backdoor attack was deployed within a Stochastic Block Model (SBM) topology. In this configuration, the network is divided into two tightly knit communities (Community A: Clients 0, 1, and 2; Community B: Clients 3, 4, and 5) connected by a sparse gateway. The malicious node (Client 1) belongs to Community A and applies the aggressive baseline boost factor of 5.0 to its local updates.

As illustrated in Figure 7, the SBM topology creates a highly complex dynamic that fundamentally alters the propagation of the attack, characterized by three distinct phenomena:

**Local Dominance:** Within the attacker’s immediate community, the malicious payload spreads effectively. The Attack Success Rate (ASR) graph shows that Client 1 quickly infects its direct neighbors. Client 0 reaches an ASR peak of nearly 0.90, while Client 2 maintains a sustained infection rate around 0.40. The high intracommunity connectivity allows the poisoned weights to circulate and reinforce themselves, creating a localized echo chamber where the backdoor thrives.

**The Topological Firewall and The Dilution Effect:** Despite the high infection rate within Community A, the attack completely fails to penetrate Community B. The ASR lines for Clients 3, 4, and 5 remain flat at 0.0 throughout all 24 communication rounds. This failure perfectly highlights a critical topological defense mechanism inherent to the SBM. This phenomenon aligns with the "dilution effect" recently identified by Palmieri et al. (2024) in the context of legitimate Decentralized Federated Learning. Their research demonstrates that when nodes are grouped in segregated communities, it is exceptionally difficult for knowledge to circulate outside those boundaries, as updates get heavily diluted during the neighborhood aggregation phase. Our results successfully translate this concept into the cybersecurity domain: we prove that this exact same structural bottleneck actively functions as a robust, natural topological firewall against malicious backdoor injections. When the gateway node



**Figure 8:** Metric per Node in a SBM topology with a boosting factor of 3.0

attempts to pass the poisoned model to Community B, the clean consensus of the receiving community immediately neutralizes the payload.

**The Cost of Aggression:** While the attacker successfully compromised its local community, this dominance came at a critical cost to its stealthiness. The top graph reveals that at round 14, the aggressive boosting factor of 5.0 causes a severe degradation in the attacker’s own Clean Accuracy, which plummets from 0.90 to approximately 0.55. By repeatedly forcing a heavily boosted malicious update against the stabilizing consensus of the network, the attacker’s local neural network suffers from Catastrophic Forgetting. In a realistic deployment, this massive deviation from the network’s average performance would immediately trigger anomaly detection protocols, leading to the isolation of the malicious node.

This failure in stealthiness suggests that an intelligent adversary must adapt. To successfully bypass anomaly detection while still attempting to cross the topological firewall, the attacker must lower its boosting factor, leading to the evaluation of the "Adaptive Attacker" trade-off.

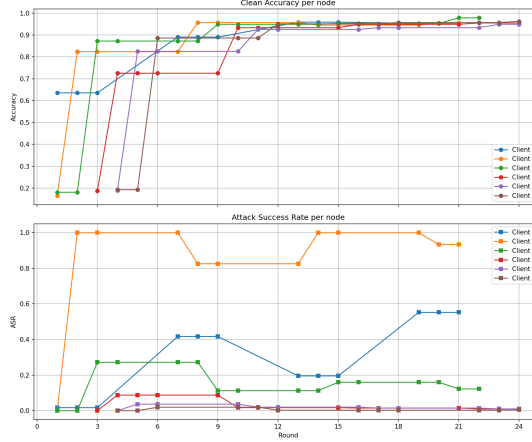
### 6.3.1 The Adaptive Attacker: Navigating the Trade-off

The catastrophic failure of the attacker’s stealth in the previous scenario (using a boost factor of  $\alpha=5.0$ ) highlights a fundamental limitation: brute force is easily detectable in structurally constrained networks. To simulate a more sophisticated adversary, we introduce the "Adaptive Attacker" model. In this scenario, the malicious node calibrates its boosting factor downward to find the optimal balance between evading anomaly detection (maintaining high Clean Accuracy) and successfully propagating the backdoor (maximizing Attack Success Rate).

To empirically locate this threshold, two subsequent simulations were executed within the same SBM topology using reduced boosting factors of  $\alpha=3.0$  and  $\alpha=4.0$ .

**The Lower Bound: Over-Prioritizing Stealth  $\alpha=3.0$ :** When the boosting factor is reduced to 3.0 (Figure 8), the attacker successfully solves the stealth problem. The top graph

## 6. EXPERIMENTAL RESULTS AND DISCUSSION



**Figure 9:** Metric per Node in a SBM topology with a boosting factor of 4.0

demonstrates that the Catastrophic Forgetting phenomenon is entirely eliminated; Client 1 maintains a Clean Accuracy of 0.95, perfectly blending in with the honest nodes. However, this stealth comes at the expense of effectiveness. The Attack Success Rate graph reveals that the poisoned payload is now too weak to dominate even its local environment. The honest updates from its immediate neighbors (Clients 0 and 2) easily wash out the backdoor, preventing the local ASR from ever surpassing 0.20. The attack is effectively rendered harmless by standard local averaging.

**The "Sweet Spot": Local Dominance ( $\alpha=4.0$ ):** By slightly increasing the boosting factor to 4.0 (Figure 9), the adaptive attacker finds its optimal local configuration.

Under this optimized configuration, the attacker successfully maintains its stealth by preserving a Clean Accuracy near 0.95, effectively evading anomaly detection. Simultaneously, the malicious payload proves potent enough to reinfect Community A. This is evidenced by Client 0's Attack Success Rate (ASR) steadily climbing past 0.55, while Client 2 sustains an ASR of approximately 0.15. Together, these metrics demonstrate that the attacker (Client 1) has successfully compromised its local cluster without triggering the catastrophic self-destruction observed at higher boosting factors.

**The Ultimate Resilience of the Topological Firewall:** The most critical finding of the adaptive attacker experiment lies in the behavior of Community B (Clients 3, 4, and 5) under the optimal attack configuration ( $\alpha=4.0$ ). Despite the attacker successfully fine-tuning its payload to be both highly infectious locally and completely stealthy, the ASR lines for Community B remain entirely flat at 0.0.

This result provides definitive proof of the structural robustness of the Stochastic Block Model. The bottleneck between communities acts as an insurmountable mathematical barrier. For the backdoor to survive the "dilution effect" across the gateway and infect Community B, the attacker would need a mathematically stronger payload (e.g.,  $\alpha=5.0$ ). However, as proven previously, applying that level of force inevitably causes Catastrophic Forgetting, exposing

the attacker. Therefore, the topology mathematically corners the adversary: it forces the attacker into an unwinnable scenario where it must choose between being ineffective or being detected.



# **Eranskina / apéndice**

Eranskinak



# Bibliography

- [1] Brendan McMahan, Ewan Moore, Daniel Ramage, Seth Hampson, and Blaise Aguera y Arcas. Communication-efficient learning of deep networks from decentralized data. *Artificial intelligence and statistics*, pages 1273–1282, 2017.
- [2] Róbert Ormándi, István Hegedűs, and Márk Jelasity. Gossip learning with l2-regularized linear models. *Parallel Computing*, 39(12):769–785, 2013.
- [3] Qingqian Yang, Peishen Yan, Xiaoyu Wu, Jiaru Zhang, Tao Song, Yang Hua, Hao Wang, Liangliang Wang, and Haibing Guan. Stealthy backdoor attack in federated learning via adaptive layer-wise gradient alignment. In *Proceedings of the IEEE/CVF International Conference on Computer Vision*, pages 29163–29172, 2025.
- [4] Georgios Syros, Gökberk Yar, Simona Boboila, Cristina Nita-Rotaru, and Alina Oprea. Backdoor attacks in peer-to-peer federated learning. *arXiv preprint arXiv:2301.09732*, 2024.
- [5] Adam Piaseczny, Eric Ruzomberka, Rohit Parasnis, and Christopher G Brinton. Adversarial node placement in decentralized federated learning: Maximum spanning-centrality strategy and performance analysis. *IEEE Internet of Things Journal*, 2025.
- [6] Ying Bao, Kebin Song, Jiyong Liu, Yan Wang, Yu Yan, and Hongkai Yu. A deep learning-based method for surface defect detection of hot-rolled steel strip. *The International Journal of Advanced Manufacturing Technology*, 102:399–405, 2019.
- [7] Lin Yuan, Zheng Wang, Lichao Sun, Philip S Yu, and Christopher G Brinton. Decentralized federated learning: A survey and perspective. *IEEE Internet of Things Journal*, 11(21):34617–34638, 2024.
- [8] István Hegedűs, Gábor Danner, and Márk Jelasity. Gossip learning as a decentralized alternative to federated learning. In *Distributed Applications and Interoperable Systems*, pages 74–90. Springer, 2019.
- [9] Paul Vanhaesebrouck, Aurélien Bellet, and Marc Tommasi. Decentralized collaborative learning of personalized models over networks. *arXiv preprint arXiv:1710.06175*, 2017.
- [10] Ehsan Hallaji, Roozbeh Razavi-Far, Mehrdad Saif, Boyu Wang, and Qiang Yang. Decentralized federated learning: A survey on security and privacy. *IEEE Transactions on Big Data*, 10(2):194–213, 2024.
- [11] Gyojin Han, Jaehyun Choi, Haehyun Lee, and Junghwan Kim. Reinforcement learning-based black-box model inversion attacks. *arXiv preprint arXiv:2304.04625*, 2023.
- [12] Nader Bouacida and Prasant Mohapatra. Vulnerabilities in federated learning. *IEEE Access*, 9:63229–63257, 2021.

## BIBLIOGRAPHY

---

- [13] Xiaofeng Xie, Chuanpu Hu, Hao Ren, and Jing Deng. A survey on vulnerability of federated learning: A learning algorithm perspective. *Neurocomputing*, 573:127225, 2024.
- [14] Guoliang Xia, Jiacheng Chen, Chunhua Yu, and Jianfeng Ma. Poisoning attacks in federated learning: A survey. *IEEE Access*, 11:10708–10722, 2023.
- [15] Eugene Bagdasaryan, Andreas Veit, Yiat Chia, Maxim Fedorov, and Deborah Estrin. How to backdoor federated learning. *International Conference on Artificial Intelligence and Statistics*, pages 2938–2948, 2020.
- [16] Robert M French. Catastrophic forgetting in connectionist networks. *Trends in cognitive sciences*, 3(4):128–135, 1999.
- [17] Daniel J Beutel, Taner Topal, Akhavan Akhavanniaz, Nicolas Marti, Yan He, Lorenzo Barkotto, and Nicholas D Lane. Flower: A friendly federated learning framework. *arXiv preprint arXiv:2007.14390*, 2020.
- [18] Luigi Palmieri, Chiara Boldrini, Lorenzo Valerio, Andrea Passarella, and Marco Conti. Impact of network topology on the performance of decentralized federated learning. *arXiv preprint arXiv:2402.18606*, 2024.
- [19] Ludovico Giaretta and Sarunas Girdzijauskas. Gossip learning: Off the beaten path. *2019 IEEE International Conference on Big Data*, pages 2273–2282, 2019.
- [20] Paul W Holland, Kathryn Blackmond Laskey, and Samuel Leinhardt. Stochastic blockmodels: First steps. *Social networks*, 5(2):109–137, 1983.
- [21] Yann LeCun, Léon Bottou, Yoshua Bengio, and Patrick Haffner. Gradient-based learning applied to document recognition. *Proceedings of the IEEE*, 86(11):2278–2324, 1998.
- [22] Diederik P Kingma and Jimmy Ba. Adam: A method for stochastic optimization. *arXiv preprint arXiv:1412.6980*, 2014.